

TECHNICAL EVALUATION EXAMINATION

Core Algorithms & Data Structures (Problem Set C)

Course / Topic: Advanced Algorithmic Design

Total Marks: 100 Marks

Duration: 180 Minutes (3 Hours)

Date: July 23, 2026

INSTRUCTIONS TO CANDIDATES:

- Answer all questions. Ensure code solutions are optimal in both time and space complexity.
- State any assumptions made clearly alongside your analysis.
- Include runtime complexity analysis (Big-O notation) for all implementations.

Question 1: Interval Scheduling & Greedy/Min-Heap Optimization

[30 Marks]

Problem: Meeting Room Allocator (Medium)

An office has several meeting requests for the day. Each meeting has a start time and an end time. Your task is to find the minimum number of conference rooms required so that no two overlapping meetings share the same room.

EXAMPLES

Input: `[[0, 30], [5, 10], [15, 20]]`
Output: `2`
Explanation: Meetings `[0,30]` and `[5,10]` overlap, so 2 rooms needed. `[15,20]` can reuse the room freed by `[5,10]`.

Input: `[[7, 10], [2, 4]]`
Output: `1`
Explanation: No overlap.

Input: `[[1, 5], [2, 6], [3, 7]]`
Output: `3`
Explanation: All three overlap at time 3.

TEST CASES REFERENCE:

#	Input	Expected Output	Notes
1	<code>[[0, 30], [5, 10], [15, 20]]</code>	2	Basic overlap
2	<code>[[7, 10], [2, 4]]</code>	1	No overlap
3	<code>[[1, 5], [2, 6], [3, 7]]</code>	3	All overlap
4	<code>[[1, 3], [3, 6]]</code>	1	Edge: meeting ends exactly when next starts
5	<code>[[5, 8]]</code>	1	Single meeting

Constraints:

- $1 \leq intervals.length \leq 10^4$
- $0 \leq start_i < end_i \leq 10^6$

Question 2: 2D Matrix Dynamic Programming

[35 Marks]

Problem: Minimum Path Sum (LeetCode 64)

Given a $m \times n$ *grid* filled with non-negative numbers, find a path from top left to bottom right, which minimizes the sum of all numbers along its path.

Note: You can only move either down or right at any point in time.

EXAMPLES

Input: `grid = [[1,3,1],[1,5,1],[4,2,1]]`

Output: 7

Explanation: Because the path 1 -> 3 -> 1 -> 1 -> 1 minimizes the sum.

Input: `grid = [[1,2,3],[4,5,6]]`

Output: 12

Constraints:

- $m == \text{grid.length}$
- $n == \text{grid}[i].length$
- $1 \leq m, n \leq 200$
- $0 \leq \text{grid}[i][j] \leq 200$

Question 3: Backtracking & Grid Search

[35 Marks]

Problem: Word Search (LeetCode 79)

Given an $m \times n$ grid of characters *board* and a string *word*, return *true* if *word* exists in the grid.

The word can be constructed from letters of sequentially adjacent cells, where adjacent cells are horizontally or vertically neighboring. The same letter cell may not be used more than once in a single word path.

EXAMPLES

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "ABCCED"
Output: true

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "SEE"
Output: true

Input: board = [["A","B","C","E"], ["S","F","C","S"], ["A","D","E","E"]], word = "ABCB"
Output: false

Constraints:

- $m == \text{board.length}$
- $n == \text{board}[i].length$
- $1 \leq m, n \leq 6$
- $1 \leq \text{word.length} \leq 15$
- *board* and *word* consist of only lowercase and uppercase English letters.